

Tile Server

On-demand single-channel raster tiles and vector labels for ML pipelines, rendered offline from a local OpenMapTiles planet file.

v1.0.0 2026-08-25 Node.js 18+

Overview

The service turns a 75 GB OpenMapTiles `.mbtiles` vector planet file into three per-tile artefacts an ML pipeline can consume directly: a greyscale map image, a class-indexed segmentation mask, and a JSON list of named features in tile-pixel coordinates.

Every tile address `z/x/y` resolves to the same geography across all three, so an image, its mask, and its labels share one coordinate frame with no transform in between. Rendering issues **zero network requests** — the bundled styles reference no glyphs and no sprites — so output is deterministic and reproducible, which matters when tiles fetched months apart end up in the same training corpus.

The three endpoints

ENDPOINT	RETURNS
<code>/tile/:z/:x/:y.png</code>	512×512 single-channel PNG. Photometric greyscale — looks like a map. The typical model input.
<code>/tile/:z/:x/:y/mask.png</code>	512×512 single-channel PNG where every pixel value is a semantic class id. The segmentation target. <code>?mode=class ambiguous raw</code>
<code>/tile/:z/:x/:y/labels.json</code>	Named features — place names, roads, water bodies — with x/y pixel coordinates in the tile's frame.

What a tile looks like



image.png — z14/8187/5448, Southwark, London. Roads, buildings and water at distinct luma levels.



mask.png — the same tile. Visually similar, but each pixel value is a class id, not a brightness.



z0/0/0 — the entire world in one tile. Web Mercator clips above $\pm 85.05^\circ$, so the poles are cut.



z13/4061/2698 — rural Warwickshire. 69% farmland, sparse buildings, only 8 labels.

Getting started

Requirements

- Node.js 18 or later
- An OpenMapTiles-schema `.mbtiles` file (vector, `format=pbf`)
- Roughly one core per concurrent render, plus two spare

Install and run

```
npm install
```

```
MBTILES_PATH=./maptiler-osm-2020-02-10-v3.11-planet.mbtiles node server.js
```

```
# Verify it came up
curl -s localhost:3000/health

# Fetch one tile of each kind
curl -o image.png    localhost:3000/tile/14/8187/5448.png
curl -o mask.png     localhost:3000/tile/14/8187/5448/mask.png
curl -s              localhost:3000/tile/14/8187/5448/labels.json | head
```

Configuration

All configuration is by environment variable; there is no config file.

VARIABLE	DEFAULT	DESCRIPTION
MBTILES_PATH	<code>./planet.mbtiles</code>	Path to the source vector tile file
OUTPUT_DIR	<code>./tiles</code>	Disk cache root
PORT	<code>3000</code>	HTTP port
WORKERS	<code>cpus - 2</code>	Render worker processes. Each holds its own map instance
TILE_SIZE	<code>512</code>	Output edge length in pixels
PNG_LEVEL	<code>1</code>	PNG compression level, 0–9. Higher is smaller and slower
MASK_SUPERSAMPLE	<code>2</code>	Masks render at this multiple of the pixel ratio, then downsample by majority vote. <code>1</code> disables it
MASK_AMBIGUOUS_THRESHOLD	<code>0.5</code>	In <code>mode=ambiguous</code> , flag a pixel when its winning class holds no more than this share of the samples
MAX_ZOOM	<code>20</code>	Highest z accepted. Above the source maxzoom the renderer overzooms
CACHE_TTL_MS	<code>0</code>	Re-render after this many ms. <code>0</code> caches forever
CACHE_MAX_SIZE	<code>unset</code>	Disk cache cap, e.g. <code>20GB</code> . Unset means unbounded growth
CACHE_SWEEP_MS	<code>30000</code>	How often the primary checks the cap and evicts

SIZING THE WORKER POOL

MapLibre GL Native is not thread-safe and each map owns a run loop, so renders are serialised *within* a process. All parallelism comes from the worker pool — `WORKERS=1` will serve one render at a time no matter how many requests arrive. The default leaves two cores for the OS, sharp's thread pool, and SQLite reads.

API reference

GET /tile/:z/:x/:y.png

Single-channel greyscale PNG. The style paints every class as a pure grey (`r == g == b`), so channel 0 is the value — it is extracted with no luma conversion and no rounding.

```
curl -o tile.png http://localhost:3000/tile/14/8187/5448.png
```

RESPONSE HEADER	MEANING
Content-Type: image/png	PNG colour type 0 — true greyscale
X-Tile-Cached: 0 1	1 if served from the disk cache without rendering
X-Tile-Channels: 1	Always 1. Decodes to (H, W), not (H, W, 3)
Cache-Control	public, max-age=31536000, immutable

GET /tile/:z/:x/:y/mask.png

Single-channel PNG where each pixel value is a semantic class id. Guaranteed to contain only legal class values. See *Output formats* for the mapping and *Mask accuracy* for why supersampling matters.

?MODE=	EVERY PIXEL LABELLED?	RENDER	USE
class	yes	10.9 ms	Default. Inference, evaluation, anything needing a complete labelling
ambiguous	no — split votes become 255	10.5 ms	Training with <code>ignore_index=255</code>
raw	no — blends as rendered	2.4 ms	Doing your own edge handling

```
curl -o mask.png "localhost:3000/tile/14/8187/5448/mask.png"
curl -o mask.png "localhost:3000/tile/14/8187/5448/mask.png?mode=ambiguous"
```

The mode is echoed back in an `X-Mask-Mode` response header, and each mode is cached separately. `?snap=0` is the original spelling of `mode=raw` and still works.

GET /tile/:z/:x/:y/labels.json

Named features read from the same vector tile the image was rendered from, in pixel coordinates relative to the tile's top-left corner.

```
curl "localhost:3000/tile/14/8187/5448/labels.json?layers=place&limit=3"
```

```
{
  "z": 14, "x": 8187, "y": 5448, "size": 512, "count": 3,
  "labels": [
    {
      "layer": "place",
      "text": "Borough",
      "class": "suburb",
      "rank": 11,
      "geometry": "point",
      "x": 374.3,
      "y": 242.3
    }
  ]
}
```

QUERY PARAM	DESCRIPTION
layers	Comma-separated subset of <code>place</code> , <code>water_name</code> , <code>transportation_name</code> , <code>poi</code> , <code>mountain_peak</code> , <code>park</code> , <code>aerodrome_label</code> . Unknown names are ignored
lang	Language for <code>name:<lang></code> , falling back to <code>name</code> . Default <code>latin</code>
limit	Keep only the N most prominent features. Results are sorted by <code>rank</code> ascending, so this takes the top N

GET /classes • GET /health

`/classes` returns the mask's value-to-name mapping as JSON — fetch it at pipeline start rather than hard-coding the table. `/classes?mode=ambiguous` adds `"ambiguous": 255`, so a pipeline can pull exactly the vocabulary of the masks it is about to request. `/health` reports worker pid, source zoom range, tile size, and resolved paths.

```
$ curl -s localhost:3000/health
{"ok":true,"pid":48213,"sourceZoom":{"min":0,"max":14},
 "tileSize":512,"channels":1,"workers":8}
```

Errors

STATUS	CONDITION
400	Invalid tile address — non-integer, <code>z</code> outside <code>0... MAX_ZOOM</code> , or <code>x / y</code> outside <code>0...2^z−1</code> . Body is <code>{"error": "invalid tile address", ...}</code>
500	Render or decode failure. Body is <code>{"error": "<message>"}</code>

A tile address that is valid but has no data — open ocean, or outside the file's coverage — is **not** an error. It renders as empty background and returns 200.

Output formats

image.png — the model input

Single channel, 8-bit. Pixel values are cartographic brightness, spread evenly across 0–255 so that no two classes collapse into each other when reduced to one channel — the failure mode of using a conventional colour map style.

FEATURE	VALUE	FEATURE	VALUE	FEATURE	VALUE
 background	255	 grass	163	 road_major	71
 ice	232	 wood	140	 railway	48
 farmland	209	 road_minor	117	 waterway	25
 landuse	186	 building	94	 water	0

THESE ARE PAINT VALUES, NOT CLASS IDS

Overlapping geometry means a pixel's value is whatever was drawn last — a road over farmland reads as road. Antialiasing produces intermediate values freely. Use `image.png` as an input feature; use `mask.png` whenever you need to know the class.

mask.png — the label map

Single channel, 8-bit, where every pixel value is a class id. Values are spaced 23 apart so that antialiased edge pixels can be resolved back to a legal class. Under `?mode=ambiguous` the reserved value **255** also appears, marking a boundary pixel the renderer declined to attribute to any one class.

CLASS	ID	CLASS	ID	CLASS	ID
background	0	grass	92	railway	184
water	23	farmland	115	road_minor	207
ice	46	landuse	138	road_major	230
wood	69	building	161	aeroway	253

WATERWAY IS IN THE IMAGE PALETTE BUT NOT THE MASK

Linear waterways are painted in `image.png` at value 25, but `waterway` is not a mask class. In the mask those pixels take whatever polygon lies beneath them, usually `background` or `landuse`. Water *polygons* are class `water` in both.

Twelve classes, ids `0`, `23`, `46`, ... `253`. To convert to contiguous indices for training, divide by 23. Fetch `/classes` at startup rather than duplicating this table in pipeline code.

labels.json — vector annotations

FIELD	TYPE	MEANING
layer	string	Source layer the feature came from
text	string	Feature name in the requested language
class	string	OpenMapTiles subclass, e.g. <code>suburb</code> , <code>motorway</code>
rank	number	Prominence. Lower is more prominent; results sort ascending
geometry	string	<code>point</code> , <code>line</code> , or <code>polygon</code>
x	number	Pixels from the tile's left edge, 0... <code>size</code>
y	number	Pixels from the tile's top edge, 0... <code>size</code>

`x` and `y` are in the same frame as the two PNGs, so `mask[int(y), int(x)]` gives the class under a label with no transform in between. Features whose anchor falls outside the tile are dropped.

Point features report their own anchor. Lines and polygons report the *midpoint* of their geometry, which is approximate — a cartographic renderer repeats a road name along the line rather than placing it once.

THESE ARE ALL NAMED FEATURES, NOT THE DRAWN ONES

The endpoint returns every named feature present in the vector data, not the subset a cartographic style would find room to draw. The Southwark tile has **1,122** labels; a conventional style renders around 25. Use `rank` and `limit` to cut it down if you want display-realistic density.

Mask accuracy

This section describes a real failure mode that silently biases class frequencies. If you train on masks, read it.

MapLibre always antialiases line geometry — `fill-antialias: false` only covers fills — so roughly one pixel in six along a road edge lands *between* two class values. Those blends are not legal class ids, so something has to resolve them.

Why snapping alone is not enough

Snapping each pixel to the nearest legal value seems sufficient, and is not. Class ids are evenly spaced, so a blend of two classes on either side of a third lands *exactly* on that third class. Mixing `water` (23) with `wood` (69) gives 46 — which reads as `ice`. No amount of nearest-value snapping can distinguish that from genuine ice, by construction.

The fix: supersample and vote

Masks are rendered at `MASK_SUPERSAMPLE` × the *pixel ratio* — same ground, more pixels — and reduced by majority vote. Each output pixel is then decided by several samples, most of which sit in a class interior rather than on an edge. Measured on `z14/8187/5448`, the tile in `01-urban-dense`, whose vector data carries only `grass` and `sand` land cover — no ice whatsoever:

SETTING	RENDER TIME	SPURIOUS ICE	RAILWAY	ROAD_MINOR
1 — snap only	4.6 ms	1326 (0.506%)	6.58%	13.81%
2 — default	10.7 ms	9 (0.003%)	2.45%	18.23%
3	19.2 ms	0 (0.000%)	2.40%	18.40%

The error is systematic, not cosmetic speckle. At supersample 1 this tile over-reports `railway` by a factor of nearly three and under-reports `road_minor` by 4.4 points — train on that and you are fitting a class-frequency bias, not the map. Supersample 3 moves both figures by under 0.2 points against the default, so 2 is close to converged. Output contains only legal class ids at every setting; the difference is whether they are the *right* ids.

Flagging boundaries instead of guessing them

Voting still assigns a class to every pixel, including ones that are a genuine 50/50 mixture. `?mode=ambiguous` instead emits `255` there — the conventional `ignore_index`, used by Cityscapes and PASCAL VOC, and clear of the class ids, which stop at 253.

It costs nothing extra: the vote already computes the sample counts, so emitting 255 on a split rather than the winner is a branch, not a second render. It is a strict refinement of `class` — every pixel it does *not* flag carries exactly the value `class` would have assigned.

The tempting shortcut — flag any pixel whose value is not a legal class id — is both incomplete and far too broad. It misses blends that land *exactly* on a legal id, which is the `ice` case above. And on a dense urban tile it discards 17.3% of pixels when supersampling shows only 2.4% of those are genuinely undecidable; the rest are ≥75% dominated by one class. Voting first cuts the flagged share to 2.65% and spreads it evenly instead of gutting the thinnest classes:

FLAGGED SHARE OF EACH CLASS	BY "ILLEGAL VALUE"	BY SPLIT VOTE
railway	41.0%	5.0%
road_minor	40.2%	5.0%
building	4.5%	1.0%
water	3.5%	0.8%

`road_minor` is only 2.3 px wide at z14 and `railway` 1.8 px, so for linear classes the antialiased edge is most of the feature. Discarding it removes precisely the boundary supervision a segmentation model most needs.

Residual error and the exact alternative

At the default setting, remaining error is roughly half a pixel along class boundaries. If you need exact boundaries, rasterise the polygons yourself directly from the vector tile, where there is no antialiasing at all — the `labels.json` path shows how the geometry is decoded.

Using it from a pipeline

Loading a tile in Python

```
import io, requests, numpy as np
from PIL import Image

BASE = "http://localhost:3000"
CLASSES = requests.get(f"{BASE}/classes").json() # name -> id

def fetch(z, x, y):
    def png(url):
        # Both PNGs are colour type 0, so these load as (H, W) uint8 --
        # no colour conversion and no alpha channel to discard.
        return np.array(Image.open(io.BytesIO(requests.get(url).content)))

    image = png(f"{BASE}/tile/{z}/{x}/{y}.png") # (512, 512)
    mask = png(f"{BASE}/tile/{z}/{x}/{y}/mask.png") # (512, 512)
    labels = requests.get(f"{BASE}/tile/{z}/{x}/{y}/labels.json").json()
    return image, mask, labels

image, mask, labels = fetch(14, 8187, 5448)

# Ids are spaced 23 apart -- divide for contiguous training indices.
target = mask // 23 # 0..11

# The label frame and the raster frame are identical, so sampling
# the class under a label needs no transform.
for item in labels["labels"][:5]:
    cls = mask[int(item["y"]), int(item["x"])]
    print(item["text"], "over", cls)
```

Choosing tiles

Convert a latitude and longitude to a tile address with the standard Web Mercator formulas:

```
import math

def tile_for(lat, lng, z):
    n = 2 ** z
    x = int((lng + 180) / 360 * n)
    lat_r = math.radians(lat)
    y = int((1 - math.log(math.tan(lat_r) + 1 / math.cos(lat_r)) / math.pi) / 2 * n)
    return z, x, y

tile_for(51.5024, -0.0908, 14) # -> (14, 8187, 5448)
```

Zoom 14 is the source maximum and the natural working level: one tile covers roughly 1.5 km at London's latitude. Above 14 the renderer overzooms — it scales a z14 parent, so tiles remain valid but carry no extra detail. Below about z10 the data is heavily generalised and buildings and minor roads simply do not exist.

Concurrency and caching

- **Just fire requests in parallel.** Concurrent requests for the same cold tile are coalesced *within a worker*, so N simultaneous callers for one address cost at most one render per worker rather than N.

- **Cache writes are atomic** — a temp file plus rename — so a reader never sees a partial tile, and several workers writing the same tile is harmless.
- **Cache hits serve at over 12,000 tiles/s.** A second epoch over the same tile set is effectively free. A style or render-setting change starts a fresh cache namespace on its own, so it will never serve you output from a configuration you have since changed.

Sample data

The `samples/` folder holds seven tiles chosen to cover different terrain, zoom levels, and edge cases. Each folder contains exactly what the API returns — `image.png`, `mask.png`, `mask_ambiguous.png`, `labels.json` — plus a `meta.json` with the tile address, the request URLs that produced it, and a class breakdown.

`mask_ambiguous.png` is a strict refinement of `mask.png` — verified across all seven samples, zero unflagged pixels differ. How much it withholds depends on how much geometry the tile carries, from 0.00% at z0 to 3.11% on the z10 city view.

The flagged share of a class tracks how present that class is. Classes holding a real share of the tile lose very little; classes surviving only as scattered slivers are withheld almost entirely — the same mechanism that suppresses the spurious `ice`:

01-URBAN-DENSE	SHARE OF TILE	FLAGGED
building	30.51%	1.0%
road_minor	18.23%	5.0%
railway	2.45%	5.0%
farmland	0.01%	94.6%
ice	0.00%	88.9%

LOW ZOOM IS THE EXCEPTION WORTH KNOWING

At z10 buildings and minor roads survive only as slivers, so `05-low-zoom` flags 94.8% of `building` and 91% of `road_minor`. If you train on low-zoom tiles, use `mask.png` or expect almost no supervision for those classes.

FOLDER	TILE	LABELS	WHAT IT EXERCISES
00-world-z0	z0/0/0	46	Entire world. 62% water, 20% ice
01-urban-dense	z14/8187/5448	1122	Southwark. Dense buildings, full road hierarchy
02-water-coast	z13/4103/2725	9	Thames estuary. Coastline edges, 33% water
03-park-green	z14/8184/5448	788	Hyde Park. Large green polygons against landuse
04-rural	z13/4061/2698	8	Warwickshire. 69% farmland, very few labels
05-low-zoom	z10/511/340	93	Greater London. Whole-city context, no building detail
06-overzoom-z16	z16/32750/21793	92	Above source maxzoom — overzoomed from a z14 parent

Inspecting a sample

```
python3 samples/load_sample.py samples/00-world-z0
```

It reports shapes, the class distribution, the flagged share per class, and the class under each label anchor. It also asserts that the ambiguous variant never disagrees with `mask.png` outside the flagged pixels.

```
00-world-z0 z0/0/0
  The entire world in one tile. Web Mercator covers latitudes ...

image  shape=(512, 512) dtype=uint8 range=[0, 255]
mask   shape=(512, 512) dtype=uint8 unique=3 values

class distribution:
  water      23  61.75%
  ice        46  19.59%
  background  0  18.66%
  unknown values: none

one-hot stack: (3, 512, 512) (3 classes present)

labels: 46 total
  place      37
  water_name  9

mask_ambiguous.png: 0.00% flagged 255 (ignore_index)
  unflagged pixels identical to mask.png: yes
    water      0.0% flagged
    ice         0.0% flagged
    background  0.0% flagged

first 5 labels (x, y are pixels from top-left):
  Antarctica  ( 256.5,  449.8) place  over ice
  Africa      ( 281.3,  239.5) place  over background
  North America ( 101.0,  171.4) place  over background
  South America ( 169.3,  286.5) place  over background
  Oceania     ( 453.0,  282.5) place  over background
```

Nothing is flagged at z0 because the style draws only fills there — no roads or railways exist in the data at that zoom, and fills are rendered with antialiasing off, so no pixel is ever a blend.

Regenerating

```
node server.js &
node samples/generate.js http://localhost:3000
```

Edit the `SAMPLES` array in `generate.js` to add locations — it takes latitude, longitude, and zoom, and works out the tile address itself.

Performance and design

Measured throughput

Ten-core machine, eight workers, dense z14 tiles over London.

MEASUREMENT	RESULT
Image render, per worker	~7 ms/tile median (4–17 ms, by feature density)
Mask render, per worker	~14 ms/tile median at supersample 2
Image throughput, 8 workers	~1,350 tiles/s cold
Mask throughput, 8 workers	~610 tiles/s cold
Cache hits	12,000–18,000 tiles/s
Label extraction	~5 ms/tile
Payload sizes	image 15–131 KB · mask 16–81 KB · labels 1–127 KB

Masks cost roughly twice what images do. If you request both at volume, the mask is your bottleneck — `MASK_SUPERSAMPLE=1` trades the accuracy described above back for speed.

Where the speed comes from

- **No symbol layers.** Text layout and collision detection dominate a styled render; dropping labels from the style is worth about 65% of render time. Labels come from the vector data instead, where they are both cheaper and more complete.
- **Channel extraction, not greyscale conversion.** Every paint colour is a pure grey, so channel 0 is already the value. Extracting it is faster than a luma conversion and lossless.
- **PNG level 1.** About 1.3 ms per tile against 2.7 ms at level 6, for roughly 50% more bytes — worth it when the consumer is on a fast link.
- **No network at render time.** Styles reference no glyphs and no sprites. Beyond determinism, this removes a class of failure where a font CDN returns an HTML error page with HTTP 200 and the renderer fails on the unexpected payload.

Why single-channel PNG

Output is PNG colour type 0 — true greyscale — so it decodes to `(H, W)` rather than `(H, W, 3)`. This requires an explicit colourspace conversion at encode time; without it, sharp promotes a single band to sRGB and writes RGB with `r == g == b`, which carries the same values in triple the channels and about 40% more bytes.

Cache size cap and eviction

Rendered tiles land under `OUTPUT_DIR` in slippy-map layout, one namespace per artefact so the mask modes never collide. Labels are not cached to disk — they are re-decoded from the vector tile each request, at ~5 ms.

```
tiles/
  image/14/8187/5448.png
  mask/14/8187/5448.png
  mask_ambiguous/14/8187/5448.png
  mask_raw/14/8187/5448.png
```

Writes are atomic — a temp file in the same directory, then a rename — so a reader never sees a partial PNG, and two workers racing on the same address simply both write identical bytes.

Set `CACHE_MAX_SIZE` to bound the disk cache. Without it the cache grows forever — at ~62 KB per artefact that is roughly 100 GB per million tiles, and a full z14 planet would be ~17 TB.

Eviction is **LRU on reads, not writes**. A tile written long ago but requested constantly is kept; one written recently and never re-read is evicted first. When the cap is exceeded the reaper sweeps down to 90% of it, so a cache sitting at the limit does not evict on every single write.

All eviction happens in the cluster primary. The workers share one directory, so if each evicted independently they would double-count the total and delete entries the others had just written. Workers instead report writes and hits to the primary, batched once a second — cache hits run at over 12,000/s, so one IPC message per hit would cost more than the read it describes.

WHY NOT ATIME

Filesystem access time looks like the obvious recency source and is not dependable: `relatime`, the Linux default, advances it only once a day, and `noatime` never does. Either would silently degrade LRU into "least recently written" and evict exactly the tiles being read most. It is used only to seed entries already on disk at startup, where it is the sole evidence available.

CONFIGURATION IS PART OF THE CACHE KEY

A tile address alone does not identify an artefact: the same `z/x/y` renders differently under a different style, tile size, PNG level, or supersample setting. Each namespace therefore carries a short hash of exactly the inputs that affect it:

```
tiles/image-a54b3a89/14/8187/5448.png
tiles/mask-87addb24/14/8187/5448.png
```

The hash covers the built style object, so editing a colour or adding a layer in `styles.js` rotates it with no bookkeeping. Inputs are tracked per artefact rather than globally, so a change only invalidates what it actually affects:

CHANGE	IMAGE	MASK	MASK_AMBIGUOUS	MASK_RAW
TILE_SIZE	new	new	new	new
PNG_LEVEL	new	new	new	new
MASK_SUPERSAMPLE	kept	new	new	kept
MASK_AMBIGUOUS_THRESHOLD	kept	kept	new	kept
source <code>.mbtiles</code>	new	new	new	new
mask palette in <code>styles.js</code>	kept	new	new	new

Superseded namespaces are reported at startup, not deleted — the output in them is still valid, and flipping a setting back should find its cache intact. With a size cap set they age out on their own, since nothing reads them and eviction is LRU.

`GET /health` reports the live state — file count, bytes, percent of cap, and total evictions, plus the namespaces in use. In cluster mode that is the primary's last broadcast, so it lags by up to one sweep interval. With no cap configured, no tracking is installed at all and the cache behaves exactly as before.

Process model

The primary process forks `WORKERS` children and restarts any that exit unexpectedly. Each worker holds its own MBTiles handle, disk cache, and renderer, and serialises renders internally because MapLibre GL Native is not thread-safe. There is no shared state between workers beyond the filesystem cache, so scaling out to more machines needs only a shared or duplicated `OUTPUT_DIR`.

Operational notes

SYMPTOM	CAUSE AND FIX
Tiles come back entirely blank	Valid address with no data — ocean or outside coverage — which is normal and returns 200. Confirm the address with <code>tile_for()</code> and check <code>/health</code> for the source zoom range.
PNGs load as (H, W, 3)	Something in the pipeline is converting on read. The server always sends colour type 0 — check for a <code>.convert("RGB")</code> in your loader.
Mask has values not in <code>/classes</code>	You are requesting <code>?snap=0</code> , which returns raw blends by design. Drop the parameter.
Cache directory keeps growing after config changes	Each configuration gets its own namespace, and old ones are kept rather than deleted so that reverting a setting finds its cache intact. The startup log names them; delete them, or set <code>CACHE_MAX_SIZE</code> and let LRU age them out.
Low-zoom tiles are slow	z0–z6 cover enormous areas and carry correspondingly more geometry. They cache like any other tile, so the cost is paid once.
Throughput is flat regardless of client concurrency	Renders serialise within a process. Raise <code>WORKERS</code> ; check <code>/health</code> to confirm the count that actually started.

Data provenance

Tiles are rendered from a fixed OpenMapTiles v3.11 planet extract dated 2020-02-10. Output is a function of that file and the bundled styles only, so the same request returns identical bytes indefinitely. Replacing the source file changes the output — delete the cache when you do, and record which extract a corpus was built from.

Coordinate conventions

- Tile addresses are **XYZ** — y increases southward. The MBTiles file stores TMS internally; the conversion is handled on lookup.
- Label coordinates are pixels from the tile's top-left, matching image and mask array indexing as `[y, x]`.
- Web Mercator covers latitudes -85.05° to 85.05° only. The poles are not representable at any zoom.